The University of Michigan
Electrical Engineering & Computer Science
EECS 281: Data Structures and Algorithms
Fall 2026

EXAM 2
KEY 1

Wednesday – Friday, November 11 – 13, 2026
10:00AM – 10:00PM (50 minute time slots)

Uniqname: _____ **Student ID:** _____

Name: _____

Uniqname of person to your left: _____

Uniqname of person to your right: _____

Honor Pledge:
"I have neither given nor received unauthorized aid on this examination,
nor have I concealed any violations of the Honor Code."

Signature: _____

Practice

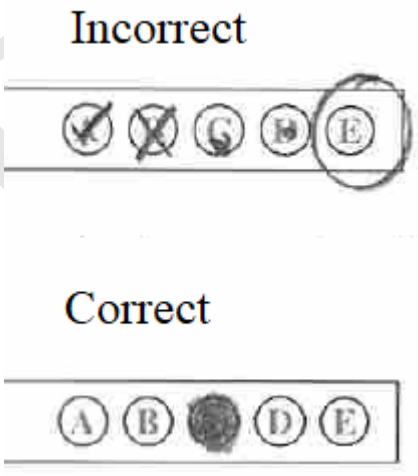
INSTRUCTIONS:

- The exam is closed book and notes except for one 8.5"x11" sheet of handwritten notes (both sides). All electronic device use is prohibited during the exam (calculators, phones, laptops, etc.).
- Print your name, student ID number, and uniqname **LEGIBLY**. Sign the Honor Pledge; this pledge applies to both the written and multiple choice sections. Your exam will not be graded without your signature.
- Record the **UNIQNAME** of the students to both your left and right. Write down `nullptr` if you are at the end of a row and there is no one on a given side.
- Record your **UNIQNAME** at the top of each odd-numbered page in the space provided. This will allow us to identify your exam if pages become separated during grading.
- This exam booklet contains **BOTH** the multiple choice and written portions of the exam. Instructions for each portion are provided at the beginning of their respective sections.
- **DO NOT** detach any pages from this booklet.
- There should be 12 pages in this book; if you are missing any pages, please let an instructor know.

Multiple Choice Portion [30 points]

MULTIPLE CHOICE INSTRUCTIONS:

- Select only **ONE** answer per multiple choice question.
- There are 12 multiple choice questions worth 2.5 points each.
- Record your choices for all multiple choice questions using the circles in the boxed area next to each question. Use a number 2 pencil, not a pen (so that you may change your answer). Fill the circle completely. There is no partial credit for multiple-choice questions. Make sure you bubble in the correct letter. See the example below for how to select your answer.



- There is no penalty for wrong answers: it is to your benefit to record an answer to each multiple-choice question, even if you're not sure what the correct answer is.
- The questions vary in difficulty — try not to spend too much time on any one question. Use your time wisely.
- When asked about memory complexity, consider all possible sources (stack and heap).

Practice

Record your answers in the bubbles next to each question.

1. Sorting

Ⓐ Ⓑ Ⓒ Ⓓ

On a nearly-sorted singly linked list, which of the following sorting algorithms is asymptotically fastest on large input?

A) Heap sort

B) Insertion sort

C) Merge sort

D) All of these would be linear

2. Quicksort

Ⓐ Ⓑ Ⓒ Ⓓ

Suppose we run quicksort on the following list [4, 12, 9, 1, 2, 5, 3, 8, 7]. We choose the last element, 7, as the pivot. After partitioning as described in lecture (but **BEFORE** we run the recursive function calls) what is the resulting list?

A) [4, 3, 5, 1, 2, 7, 12, 9, 8]

B) [4, 3, 5, 1, 2, 9, 12, 8, 7]

C) [4, 3, 1, 5, 2, 7, 12, 8, 9]

D) [4, 3, 5, 1, 2, 7, 12, 8, 9]

3. Ordered Arrays

Ⓐ Ⓑ Ⓒ Ⓓ Ⓔ

Given two sorted arrays, each with n numbers, you are asked to find the median of all of the elements from the two sorted arrays. What is the time complexity of the best solution?

A) $O(1)$

B) $O(\log n)$

C) $O(n)$

D) $O(n \log n)$

E) $O(n^2)$

Practice

Record your answers in the bubbles next to each question.

4. Collision Resolution

Ⓐ Ⓑ Ⓒ Ⓓ

You are given a hash table with size $M = 10$ and hash function $H(n) = (2n + 3) \mod M$. Collisions are resolved using **quadratic** probing. What would the table look like (with an X denoting an empty location) after inserting the following elements in the order given? Use $c1 = 0$ and $c2 = 1$.

4, 10, 12, 7, 2

A) [X, 4, X, 10, X, X, X, 12, 7, 2]

B) [X, 4, 2, 10, X, X, X, 12, 7, X]

C) [X, 4, X, 10, X, X, 2, 12, 7, X]

D) [X, 4, X, 10, X, X, 2, 12, X, 7]

5. Hash Table Applications

Ⓐ Ⓑ Ⓒ Ⓓ

In which one of the following situations would you want to use a hash table?

A) You have a classroom full of students each with a 10-digit ID number, and you want to find them by their ID number.

B) You want to find the highest priority thread to execute, each with its own assigned priority.

C) You have a rectangular floor composed of square tiles, and you want to know the color of a tile given a set of coordinates.

D) You are given a set of names, and want to print them out in alphabetical order.

6. Inorder Successor

Ⓐ Ⓑ Ⓒ Ⓓ Ⓔ

Suppose that in a binary search tree, when a node with two children is deleted, it is replaced by its inorder successor. Given a pointer to the node to be deleted, what is the time complexity of finding the inorder successor in the average case, if the tree contains n nodes?

A) $\Theta(\log n)$

B) $\Theta(n)$

C) $\Theta(n \log n)$

D) $\Theta(n^2)$

E) $\Theta(n!)$

Practice

Record your answers in the bubbles next to each question.

7. Tree ADT

Ⓐ Ⓑ Ⓒ Ⓓ Ⓔ

What does the following function return?

```
1       int count(Node* curNode) {
2         if (curNode == nullptr)
3           return 0;
4         else if (curNode->parent == nullptr)
5           return 1 + count(curNode->left) + count(curNode->right);
6         else if (curNode->left == nullptr && curNode->right == nullptr)
7           return 1;
8         else
9           return count(curNode->left) + count(curNode->right);
10      } // count()
```

A) The number of nodes in the tree

B) The number of internal nodes

C) The number of external (leaf) nodes

D) The number of external (leaf) nodes plus the root

E) The depth of the tree

8. Tree Traversal

Ⓐ Ⓑ Ⓒ Ⓓ Ⓔ

Given the following preorder and inorder traversals of a binary tree, what is the postorder traversal of this same binary tree?

- Preorder: 11, 19, 31, 7, 10
- Inorder: 31, 19, 11, 10, 7

A) 11, 19, 7, 31, 10

B) 31, 19, 10, 7, 11

C) 19, 31, 10, 7, 11

D) 31, 10, 7, 19, 11

E) 10, 7, 31, 19, 11

Practice

Record your answers in the bubbles next to each question.

9. Binary Search Tree Complexity

Ⓐ Ⓑ Ⓒ Ⓓ Ⓔ

Suppose you have a binary search tree, where h represents the tree's height and n represents the number of nodes. What is the worst-case complexity of a search operation on this tree in terms of h and/or n?

A) $\Theta(n \log h)$

B) $\Theta(n^2)$

C) $\Theta(\log n)$

D) $\Theta(h)$

E) $\Theta(n \log_n h)$

10. AVL Trees

Ⓐ Ⓑ Ⓒ Ⓓ Ⓔ

Which of the following statements about AVL trees is **TRUE**?

A) Optimally re-balancing an AVL tree after a deletion can take more than $O(1)$ time.

B) For an insert operation, up to three rotations are needed.

C) Searching an AVL tree can take longer than $O(\log n)$.

D) Given an n-node binary search tree that has never been balanced, the BST can be made into a balanced AVL tree using $O(\log n)$ rotations.

E) A tree is AVL balanced if and only if the absolute difference in heights of the root's children is less than 2.

11. Sorting Linked Lists

Ⓐ Ⓑ Ⓒ Ⓓ Ⓔ

If you modified a sorting algorithm to sort a linked list, which one of the following sorts would have its average-case complexity get worse?

A) Insertion sort

B) Mergesort

C) Quicksort

D) Bubble sort

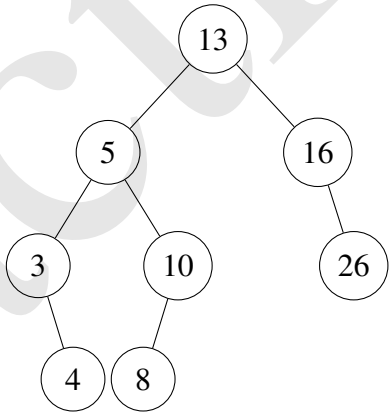
E) All of them would keep their same average-case complexity

Practice

Record your answers in the bubbles next to each question.

12. Insertion/Deletion Ⓐ Ⓑ Ⓒ Ⓓ Ⓔ

Given the following AVL tree:



Perform the following operations:

1. Delete 13 (replace with its inorder successor)
2. Insert 9

What are the left and right children of "16" after performing the operations?

A) 8, 26

B) 9, 26

C) 10, 26

D) `nullptr`, 26

E) `nullptr`, `nullptr`

Practice

Written Portion [20 points]

WRITTEN PORTION INSTRUCTIONS:

- There are **two** questions in this section, worth 20 points.
- Please write your solution legibly in the space provided. Solutions that are difficult to read may receive fewer points.
- Use the back of the question page to work out solutions for each problem and then rewrite your final answer in the space provided.
- The directions for each problem will specify which STL algorithms/functions may be used in your solution.
- Solutions that exceed the allowed line count will lose one (1) point per line over the limit.
- Partial credit will be awarded for partial solutions.
- Errors in program logic or syntax may result in a reduced score.
- Be sure to consider all possible sources of memory (stack and heap) and time complexity.

Practice

13. Programming: Searching in a Tree [8 Points]

A *set* is a data structure that can efficiently (less than $O(n)$ time) see if it contains a given element. Using the following definition of a binary tree node:

```
struct Node {
  int val;
  Node *left;
  Node *right;
};
```

Implement the `exists()` function on a BST-based set.

Requirements: In the average case, your solution must be $O(\log n)$, and you may use up to $O(\log n)$ auxiliary space.

Implementation: Use the back of this page as a working area, then rewrite **neatly** on the front. Limit: 15 lines of code (points deducted if longer). You may use any C, C++, or STL function/algorithm that you wish.

```
bool exists(Node *node, int val) {
```

Practice

This page is intentionally left blank.
You may use this page as working space.

Practice

14. Programming: Searching in a Hash Table [12 Points]

A *set* is a data structure that can efficiently (less than $O(n)$ time) see if it contains a given element. Given the following definition of a hash table structure:

```
enum class Status { Empty, Occupied, Deleted };
struct HashTable {
  std::vector<int> buckets;
  std::vector<Status> status;
};
```

Suppose this hash table implements the quadratic probing mechanism for collision resolution. When hashing an integer value `val`, the hash function is simply `val % M`, where `M` represents the size of the hash table. Implement the `exists()` function for a set based on a hash table. Use $c1 = 1$ and $c2 = 2$.

Requirements: Your solution must implement quadratic probing. Your solution must be $O(n)$ time. You may use $O(1)$ auxiliary space.

Implementation: Use the back of this page as a working area, then rewrite **neatly** on the front. Limit: 16 lines of code (points deducted if longer). You may use any C, C++, or STL function/algorithm that you wish.

```
bool exists(const HashTable &tbl, int val) {
```

Practice

This page is intentionally left blank.
You may use this page as working space.

Practice